

1. **Exercício:** Escreva uma função em Python que receba dois números inteiros. Se o produto dos dois números for menor ou igual a 1000, retorne o produto; caso contrário, retorne a soma. **Objetivo do exercício:** Aprender os conceitos básicos de fluxo de controle e o uso de instruções if-else. Compreender como as decisões do código alteram a saída com base em um limite matemático.

Entrada fornecida:

- Caso 1: `number1 = 20,number2 = 30`
- Caso 2: `number1 = 40,number2 = 30`

2.Exercício: Percorra os 10 primeiros números (0–9). Em cada iteração, imprima o número atual, o número anterior e a soma deles. **Objetivo do exercício:** Este exercício ensina o conceito de "rastreamento de estado". Em programação, muitas vezes é necessário armazenar um valor de uma iteração anterior de um loop para calcular os resultados na iteração atual. Essa é a base de algoritmos como as sequências de Fibonacci ou totais acumulados.

Entrada fornecida: Intervalo: `numbers = range(10)`

3.Exercício: Exiba apenas os caracteres que estão presentes em um índice par na string fornecida.

Objetivo do exercício: Compreender como os dados são armazenados na memória usando indexação baseada em zero. Na maioria das linguagens, o primeiro caractere está na posição 0, o segundo na posição 1 e assim por diante. Dominar a indexação é fundamental para a análise sintática de dados.

Entrada fornecida: String:"pynative"

4.Exercício: Escreva uma função para remover caracteres de uma string, começando no índice 0 até o índice n, e retorne uma nova string.

Objetivo do exercício: Este exercício demonstra como truncar cadeias de dados, uma tarefa comum de limpeza de dados.

Entrada fornecida:

- `remove_chars("pynative", 4)`
- `remove_chars("pynative", 2)`

5.Problema prático: Escreva um programa para trocar os valores de duas variáveis, `a` e `b`, sem usar uma terceira variável temporária.

Objetivo do exercício: Este exercício ajudará você a aprender sobre eficiência de memória e o recurso especial de desempacotamento de tuplas do Python. Em outras linguagens, como C ou Java, você precisa de uma variável temporária para trocar valores com segurança. Em Python, você pode trocar valores em uma única linha sem correr o risco de perder dados.

Entrada fornecida: `a = 5, b = 10`

6.Exercício: Escreva um programa que calcule o fatorial de um número dado (ex.: 5!) usando um `for` laço de repetição.

Objetivo do exercício: Este exercício explora a "Acumulação Matemática". Um fatorial (por exemplo, $5! = 5*4*3*2*1$) exige que você mantenha um produto cumulativo ao longo de múltiplas iterações, o que é um padrão fundamental na computação científica.

Entrada fornecida :`number = 5`

7.Exercício: Crie uma lista de 5 frutas. Adicione uma nova fruta ao final da lista e, em seguida, remova a segunda fruta (no índice 1).

Objetivo do exercício: Este exercício ensina "Gerenciamento dinâmico de coleções". Listas raramente são estáticas; ser capaz de modificá-las, expandi-las e limpá-las é essencial para lidar com dados como carrinhos de compras, listas de usuários ou sistemas de estoque.

Entrada fornecida: `fruits = ["apple", "banana", "cherry", "date", "elderberry"]`

Resultado esperado: `['apple', 'cherry', 'date', 'elderberry', 'fig']`

8.Exercício: Escreva um programa que receba uma string e a inverta (por exemplo, "Python" se torna "nohtyP").

Objetivo do exercício: Este exercício demonstra o "fatiamento de sequências". Strings em Python são sequências, e dominar a sintaxe de fatiamento é um atalho poderoso para manipulação de dados que exigiria de 5 a 10 linhas de código em outras linguagens.

Entrada fornecida: `text = "Python"`

Resultado esperado: `Reversed: nohtyP`

9.Exercício: Escreva um programa para contar o número total de vogais (a, e, i, o, u) presentes em uma determinada frase.

Objetivo do exercício: Este exercício apresenta o conceito de "Teste de Pertinência". Ao verificar se um caractere pertence a um grupo específico (as vogais), você aprenderá a filtrar dados com base em categorias. Este é um passo fundamental para a criação de ferramentas de análise de texto ou filtros de spam.

Entrada fornecida :`sentence = "Learning Python is fun!"`

Resultado esperado : Número de vogais: 6

10.Exercício: Crie um cronômetro regressivo que comece em um número dado e faça a contagem regressiva até zero usando um `while` laço de repetição.

Objetivo do exercício: Neste exercício, você aprenderá sobre lógica de término de loops e gerenciamento de atrasos de tempo. Saber como controlar o fluxo do seu código em tempo real é importante para criar animações, loops de jogos ou scripts automatizados.

Entrada fornecida: `start_count = 5`

Resultado esperado: `5 4 3 2 1 Blast off!`

11.Exercício: Escreva um programa que crie um novo arquivo de texto chamado `notes.txt`, escreva três linhas de texto separadas nele e, em seguida, leia esse arquivo de volta para exibir o conteúdo no console.

Objetivo do exercício: Este exercício apresenta o conceito de "Armazenamento Persistente". Ao contrário das variáveis, que desaparecem quando o programa é encerrado, os arquivos permitem salvar dados no disco rígido. Aprender o `open()` fluxo de trabalho `write()` dos `read()` arquivos é

essencial para a criação de sistemas de registro de logs e para o salvamento das configurações do usuário.

Entrada fornecida: Linhas a escrever:

1. "Olá, esta é minha primeira mensagem."
2. "O manuseio de arquivos em Python é simples."
3. "Fim do arquivo."

Resultado esperado:

```
notes.txt
```

```
Olá, esta é minha primeira anotação.
```

```
Manipular arquivos em Python é simples.
```

```
Fim do arquivo.
```

12.Exercício: Escreva um script que abra um `.txt` arquivo existente e conte o número total de palavras que ele contém.

Objetivo do exercício: Este exercício ensina "Análise de Dados". Em ambientes profissionais, raramente trabalhamos com dados que digitamos no código; quase sempre extraímos dados de fontes externas. Este script simula técnicas básicas de mineração de texto usadas para analisar documentos ou registros.

Entrada fornecida:

Um arquivo externo `sample.txt` contendo: *"A programação é a linguagem do futuro."*

Resultado esperado: `The file contains 7 words.]`

13.Exercício: Crie uma `Car` classe com atributos para `make`, `model`, e `year`. Inclua um método chamado `start_engine()` que imprime uma string formatada descrevendo a inicialização do carro.

Objetivo do exercício: Este exercício introduz a **Programação Orientada a Objetos (POO)** . Em vez de apenas escrever funções, você criará um "Projeto" (a Classe) para gerar "Objetos" (os carros específicos). É assim que o software moderno é construído, permitindo que você organize o código em componentes lógicos e reutilizáveis.

Entrada fornecida: Marca: "Toyota", Modelo: "Camry", Ano: 2022

Resultado esperado: `The 2022 Toyota Camry's engine is now running!`

14.Exercício: Escreva um programa para verificar se um determinado número é um palíndromo. Um número palíndromo permanece o mesmo quando seus dígitos são invertidos (ex.: 121, 545).

Objetivo do exercício: Este exercício ensina "Inversão Algorítmica". Embora seja fácil inverter strings em Python, inverter um número matematicamente usando os operadores de módulo (`%`) e divisão inteira (`//`) aprofunda a compreensão de como os inteiros são armazenados na memória e como manipular dígitos individualmente.

Entrada fornecida: `number = 121`

Resultado esperado :

Número original 121

Sim. O número dado é um número palíndromo.